CSE 2421

Intro to Pointers

Machine Memory

- Computer memory consists of millions of bits
  - A bit is a single 0 or 1
- Memory is grouped into addressable groups called bytes
  - 1 byte = 8 bits
- For larger values you can address larger chunks (verbiage for a 32-bit system)
  - 1 word = 2 bytes
  - 1 doubleword = dword = 4 bytes
  - 1 quadword = qword = 8 bytes

Address Versus Contents

Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	883

- The above table contains five 32-bit integer values
- Assuming these were values in our program remembering where they are in memory would be difficult
- That's why we use variables to associate a name with a memory location

Implications of the Pointer-type

- The CPU *does not* make any assumptions about how to interpret any piece of memory, you must explicitly tell it how to do so
- The compiler does all pointer arithmetic using the guidance you gave it via the type you declared that the pointer points to
- The compiler chooses machine instructions to generate for the CPU based upon the type you declared that the pointer points to

Address Contents

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	883

```
int A = 112;
printf("%d", A);
printf("%f", A); //INVALID
printf("%s", A); //INVALID
```

If not using memory as the form is was declared at the compiler will attempt to protect you and throw an error. Must explicitly cast to use as the non-declared type

Address Contents

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	883

```
int A = 112;
printf("%d", (int)A);
printf("%f", (float)A);
printf("%s", (char)A);
```

These will all work! You tell the system how to interpret memory

Pointer definition

- Values of variables are stored in memory, at a particular location
- A location is identified and referenced with an address
  - Analogous to identifying a house's location via an address
- A pointer is a variable that contains the address of another variable
- * is used in the declaration of a pointer type
  - int *p means variable p is a pointer that points to an integer
- & (unary operator) gives the "address of" an object
  - p = &c means p is the address of c

example

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	x104	00	883

```
int *c = &b;
```

read as: c is a pointer that points to integer b

Pointers cont.

- Every pointer points to a specific data type
  - Exception = void (a generic pointer); pointer to void holds any type of pointer but can't be dereferenced (i.e. cannot get the "contents of") without casting

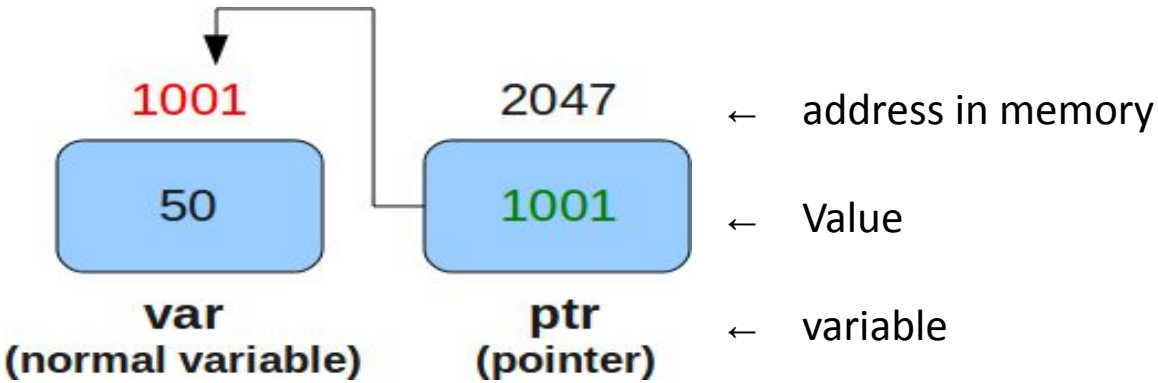
Pointer examples:

```
int *p;
char *c;
void *x;
```

Pointers Defined cont.

- `int *x;`
- Read as: declare x as a pointer to a 32-bit integer
- Interpretation: declare x as a variable on the stack that holds the numeric address of the location in memory at which are 32 bits that we intend to manipulate as a signed integer

Pointer introduction




```
int   var = 50;    //
int  *ptr;         //
ptr = &var;        // *ptr == var (set ptr to address of var)
*ptr = 1;          // var == 1 (de-refrence ptr and set that location)
```

Pointers - Sidenote

- Easily test the bitness of a machine
- `int *x;`
- sizeof(x) is the word-size of the machine in bytes
  - on a 32 bit machine that can address 2^32 bytes of memory, 4
  - on a 64 bit machine, 8
  - etc
- sizeof(*x) is 4, or sizeof(int)

Pointers cont.

- There are two fundamental ways to manipulate or obtain (dereference) memory through this pointer:
- `*x`
- Read as: dereference `x`
- Interpretation:
  - set(if left of equals)
  - get(if right of equals)

32 bits of memory interpreted as a signed integer beginning at the memory address contained in variable `x`

Pointers Cont.

- * (unary not arithmetic operator) is a dereferencing operator when applied to pointers
  - When applied to a pointer, it accesses the object the pointer points to
  - * in front of a pointer variable means “get the value at that address” i.e. “contents of”
  - int a = *p means get the value at the address designated by p and assign it to a
  - *p = 1 means assign the value of 1 to the memory location designated by the address of p

Declaring Pointers

- int* ptr_a;
- int *ptr_a;
- Both are valid, but the first style leads to mistakes
  - int* ptr_b, ptr_c, ptr_d
    - b is a pointer but c and d are integers
  - int *ptr_b, *ptr_c, *ptr_d
    - 3 pointers are declared here

Common Mistake

```
int *a;
*a = 12;
```

Whats wrong? You declared a to be a pointer to an int, but you never gave it an address to point to

Common Mistake

```
int *a;
*a = 12;
```

What happens? Unpredictable. C does not auto initialize memory to 0's so the value at *a* is unpredictable. The value will be interpreted as an address and 12 assigned to that address if possible

Common Mistake

```
int *a;
*a = 12;
```

ex: a = 0xFFFFFFFF => segfault, this isn't a valid address

ex: a = 0x7c580 => A valid address. The value at 0x7c580 gets set to 12. Now you have inadvertently overwritten something potentially important

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	1
y	2	2
z[0]	3	0
z[1]	4	0
z[2]	5	0

* not going to worry about "size" right now

// Initialize two ints and an array

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	1
y	2	2
z[0]	3	0
z[1]	4	0
z[2]	5	0
ip	6	??

* not going to worry about "size" right now

// Initialize a pointer but don't assign it a value

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	1
y	2	2
z[0]	3	0
z[1]	4	0
z[2]	5	0
ip	6	1 (addr of x)

* not going to worry about "size" right now

// Set value (memory) of ip to address of x

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	1
y	2	1
z[0]	3	0
z[1]	4	0
z[2]	5	0
ip	6	1 (addr of x)

* not going to worry about "size" right now

// Set value (memory) of y to value of derefrenced ip

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	0
y	2	1
z[0]	3	0
z[1]	4	0
z[2]	5	0
ip	6	1 (addr of x)

* not going to worry about "size" right now

// Set value of *ip to 0
// *ip is a pointer so it gets derefrenced first

Pointer example

Reminders:

- *** in a declaration says "I am a pointer" that points to a certain type of value**
- **& = "address of"**
- *** = "get the value at that address"**

EXAMPLE:

```
int x=1, y=2, z[3];
int *ip;
ip = &x;
y = *ip;
*ip = 0;
ip = &z[0];
```

VARIABLE	ADDRESS *	MEMORY
x	1	0
y	2	1
z[0]	3	0
z[1]	4	0
z[2]	5	0
ip	6	3 (addr of z[0])

* not going to worry about "size" right now

// Set ip (value) to address of z

Pointers cont.

- Ex: y = *ip + 1 takes whatever ip points at, adds 1, and assigns the result to y
- Other ways to increment by 1:
  - *ip += 1 ➔ *ip = *ip + 1
  - ++*ip
  - (*ip)++
    - The parentheses are necessary; without them, the expression would increment ip instead of what it points to, because unary operators like * and ++ associate right to left.
- Pointers are variables so can be used without dereferencing.
  - Example: int x, *iq, *ip=&x;
  - iq = ip;
    - copies the contents of ip (an address) into iq, making iq point to whatever ip points to

Pointers to Pointers

- `int **x;`
- Read as: declare x as a pointer to a pointer to a 32-bit integer
- Interpretation: declare x as a variable on the stack that holds the numeric address at which is another numeric address at which are 32 bits that we intend to manipulate as a signed integer

Double Pointer Ex:

```
int A = 112;
int *C = &A;
int **E = &C;

//E is now a double pointer
```

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	100	00	108

Double pointers?

```
// ex. modifying a pointer within a function

void allocate(int** p)
{
  *p = (int*)malloc(sizeof(int));
}

int main()
{
  int* p = NULL; // blank pointer
  allocate(&p); // Caller must free p
  *p = 42;
  free(p);
}
```

Pointer Casting

- `int i = 5;`
- `char *y = (char *)(&i);`
- Declares i as an integer on the stack and puts 5 in that location, then declares y as a pointer to a character and assigns the address of i casted (interpreted as) to a pointer to a character to it.
- &i and y both have the same numeric value pointing to the same piece of memory
- Dereferencing y hereafter generates instructions that operate on chars at i's memory address instead of integers (compiler magic)

Casting Example

```
enum integral_type
{
    ITYPE_CHAR = 0,
    ITYPE_SHORT,
    ITYPE_INT,
    ITYPE_LONGLONG
};
void doublevalue(void *ptr_to_integral_type, enum integral_type data_type)
{
    if(ptr_to_integral_type == NULL)
        return;
    switch(data_type)
    {
    case ITYPE_CHAR: // 8-bit value at pointer
        *((char*)ptr_to_integral_type) *= 2;
        break;
```

Casting Example

```
    // Continued from previous
    case ITYPE_SHORT: // 16-bit value at pointer
        *((short *)ptr_to_integral_type) *= 2;
        break;
    case ITYPE_INT: // 32-bit value at pointer
        *((int *)ptr_to_integral_type) *= 2;
        break;
    case ITYPE_LONGLONG: // 64-bit value at pointer
        *((long long*)ptr_to_integral_type) *= 2;
        break;
    default:
        return;
    }
}
```

Pointer examples… more!

- Every pointer points to a specific data type.
  - one exception: “pointer to void” is used to hold any type of pointer but cannot be dereferenced itself (later)
- If ip points to the integer x, (ip=&x) then *ip can occur in any context where x could
  - Example: *ip = *ip + 10 ➔ x=x+10; increments the contents of the address at ip by 10
- The unary operators * and & bind more tightly than arithmetic operators

RECAP

- In a declaration
  - * says “I am a pointer” that points to a certain type of value
- In a statement
  - & = “get the address of variable”
  - * = “get the value at that address”

Exercise 1

```c
#include<stdio.h>
int main() {
    float i=10, *j;
    void *k;
    k=&i;
    j=k;
    printf("%f\n", *j);
    return 0;
}
```

Prints:

10.000000

Exercise 2

```
#include <stdio.h>
#include <stdlib.h>
main() {
    int x, *p;
    p = &x;
    *p = 0;
    printf("x is %d\n", x);
    printf("*p is %d\n", *p);
    *p += 1;
    printf("x is %d\n", x);
    (*p)++;
    printf("x is %d\n", x);
    return 0;
}
```

x is 0
p is 0

x is 1
x is 2

Exercise 3

```c
#include <stdio.h>
int main(void) {
    char ch = 'c';
    char *chptr = &ch;
    int i = 20;
    int *intptr = &i;
    float f = 1.20000;
    float *fptr = &f;
    char *ptr = "I am a string";
    printf("\n [%c], [%d], [%f], [%c], [%s]\n",
        *chptr, *intptr, *fptr, *ptr,  ptr);
    return 0;
}
```

[c], [20], [1.200000], [I], [I am a string]

R and L values

- L-value = something that can appear on the left side of an equal sign
  - A place i.e. memory location for a value to be stored
- R-value is something that can appear on the right side of an equal sign
  - A value
- Example:
  - a = b+25 vs b+25 = a

Ex

```
int a;
int *d = &a;

*d = 10-*d
```

//R-obtain the value in the location to which d points (the value of a) and subtract it from 10
//L-assign to the location to which d points (value of a)

Ex cont.

```
int a;
int *d = &a;

d = 10-*d  //BAD IDEA
```

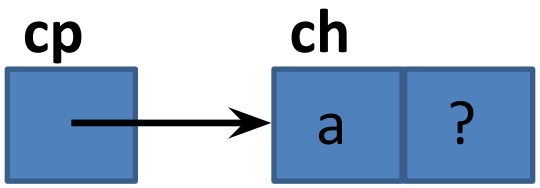
//R-obtain the value in the location to which d points (the value of a) and subtract it from 10
//L-assign to the value of d

This will generate a warning in the compiler that we're trying to use a variable in context that is inconsistent with its type.

Don't ignore warnings!

R and L values (cont)

- Given:
  - char ch = ‘a’;
  - char *cp = &ch;
- NOTE: the ? is the location that follows ch



Problem	Expression	L-value	R-value
1	ch	yes	yes
2	&ch	illegal	yes
3	cp	yes	yes
4	&cp	illegal	yes
5	*cp	yes	yes
6	*c+1	illegal	yes
7	*(c+1)	yes	yes
8	++cp	illegal	yes
9	cp++	illegal	yes
10	*++cp	yes	yes
11	*cp++	yes	yes
12	++*cp	illegal	yes
13	(*cp)++	illegal	yes
14	++*++cp	illegal	yes
15	++*cp++	illegal	yes

Pointer fun

*&a = 25 // What does this do?

Unary operators work from R -> L

The & operator generates the address where variable a is stored

The * operator goes to the location whose address is given as the operand. At this point the operand is the address of a

Effectively assigning the value of 25 to a
a = 25

Pointer fun ex2

assume variable B is stored at location 500
*500= 13

What does that do?

Nothing! Its invalid. The * operator must be used on a pointer expression

Pointer fun ex2 cont.

- But what if we really want to assign to direct memory addresses?

```
*(int *)500 = 13
```

OR

```
int * x;
x = (int*)500;
*x = 13;
```

Last Words

- Don’t Panic, pointers are crazy!